11.2 Compound queries

We worked with leaf queries in the last two chapters (term-level and full-text queries): queries that work on single (individual) fields. If our requirement is to find the top-selling books from a period, we can use a leaf query to fetch those results. Leaf queries help find answers to simple questions without support for conditional clauses. However, the real world seldom involves simple query requirements.

Most requirements demand developing complex queries with multiple clauses and conditions. For example, a complex query may consist of finding best-selling books written by a particular author and published during a specific period, or a particular edition; or returning all books categorized by various geographical zones except a specific country, ordered by the highest grosser.

Here's where compound queries shine: they help develop complex search queries by combining one or more leaf queries. Fortunately, we can use Query DSL (discussed in chapter 8) to write compound queries: we use the same `_search` endpoint with a request body that consists of a `query` object. Figure 11.1 shows the syntax for a compound query.

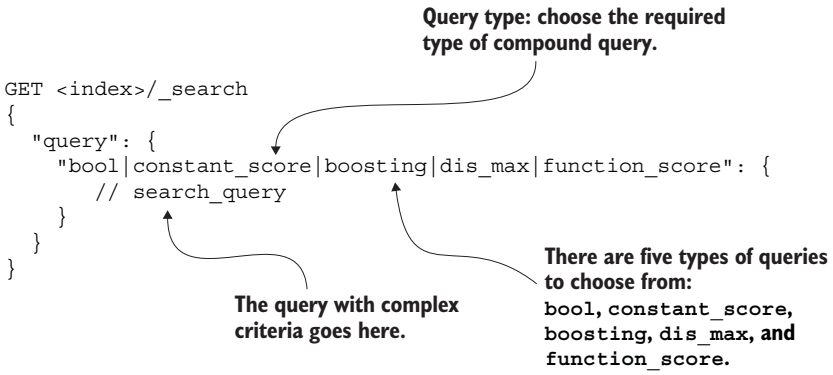


Figure 11.1 Compound query syntax

The basic syntax for compound queries is no different from other queries. However, the body of the `query` object is made of different components, depending on the type of compound query we want to use.

Elasticsearch provides five such queries for varied search requirements: Boolean (`bool`), constant score, function score, boosting, and disjunction max. Table 11.1 briefly describes these five compound queries. For example, if our requirement is to develop an advanced query using conditional clauses, we can use a Boolean query encompassing multiple leaf queries using `AND`, `OR`, and other conditions. Similarly, if the requirement is to set a static score across all results, a constant-score query is what we need. We apply these five types of queries in different use cases in this chapter.